

SpacêFarm — Cahier des charges

Mini-serre connectée pour la culture de tomates — Arduino Yún + Raspberry Pi 3

Sep 24, 2026 · @Marie-CLau

Contexte et idée

SpacêFarm est une mini-serre instrumentée qui mesure en continu l'environnement d'un plant de tomate, journalise ces mesures et permet d'agir à distance sur l'arrosage et l'éclairage.

Le constat de départ est simple : une plante d'intérieur meurt rarement d'un manque de soin, mais d'un soin mal dosé. On arrose trop, ou au mauvais moment, et on n'a aucune trace de ce qui s'est réellement passé entre deux observations. Le jardinier amateur n'a ni historique, ni indicateur objectif.

Le projet répond à ce manque par une chaîne complète, du capteur à la base de données : trois grandeurs mesurées (température, humidité de l'air, luminosité), un niveau d'eau surveillé, un arrosage déclenché à la demande et tracé, un éclairage de croissance commandé. Le tout tient dans un boîtier imprimé en 3D conçu pour l'occasion, posé à côté du pot.

Le projet s'adresse à un usage domestique ou pédagogique. Il sert aussi de démonstrateur d'une chaîne IoT complète : acquisition embarquée, transport réseau, persistance et restitution.

Objectifs et périmètre

L'objectif du rendu est un prototype fonctionnel de bout en bout, pas une maquette. Une mesure prise par un capteur doit se retrouver dans la base, et une commande envoyée depuis le Raspberry Pi doit faire tourner la pompe.

Dans le périmètre

- Mesure de la température et de l'humidité de l'air (DHT11)
- Mesure de la luminosité ambiante (photorésistance + diviseur 10 k Ω)
- Mesure du niveau d'eau du réservoir (module résistif)
- Journalisation horodatée de toutes les mesures en base SQLite
- **Dashboard web hébergé sur le Raspberry Pi** : centre de contrôle accessible par un lien, affichant les mesures en direct et l'historique
- Arrosage commandé depuis le dashboard, borné en durée et tracé en base

- Éclairage de croissance commandé depuis le dashboard (LED rouge/bleu)
- Boîtier et supports conçus puis imprimés en 3D

Hors périmètre

- Régulation automatique en boucle fermée : l'arrosage reste commandé, pas déclenché par un algorithme décidant seul quand la plante a soif
- Mesure de l'humidité du substrat : le capteur capacitif n'est pas dans la nomenclature actuelle
- Application mobile
- Hébergement distant ou accès depuis l'extérieur du réseau local
- Toute preuve agronomique de l'effet de l'éclairage sur la croissance : la durée du projet ne permet pas d'observer une pousse

Cette délimitation est volontaire. Une chaîne complète et fiable sur un périmètre restreint vaut mieux qu'un périmètre large à moitié câblé le jour du rendu.

Fonctions attendues

Chaque fonction porte un critère d'acceptation vérifiable : c'est ce qui sera démontré au rendu.

Réf.	Fonction	Critère d'acceptation	Priorité
F1	Mesurer température et humidité de l'air	Une valeur cohérente toutes les 60 s, écart < 2 °C avec un thermomètre de référence	Indispensable
F2	Mesurer la luminosité ambiante	La valeur varie nettement entre plein jour et main posée sur le capteur	Indispensable
F3	Mesurer le niveau d'eau du réservoir	Distingue réservoir vide, à moitié et plein	Indispensable
F4	Journaliser les mesures en base	Chaque mesure devient une ligne horodatée dans SQLite ; aucune perte lors d'une coupure de liaison de 5 min	Indispensable
F5	Arroser à la demande	Une commande déclenche la pompe, qui s'arrête seule à la durée demandée et au plus tard à 20 s	Indispensable

Réf.	Fonction	Critère d'acceptation	Priorité
F6	Commander l'éclairage de croissance	Allumage et extinction à la demande ; état lisible à tout moment	Indispensable
F7	Superviser depuis un dashboard web	Un lien ouvre une page affichant les dernières mesures et les courbes des 24 h, depuis n'importe quel appareil du réseau local	Indispensable
F8	Commander les actionneurs depuis le dashboard	Un bouton déclenche l'arrosage, un autre l'éclairage, avec retour visuel de l'état	Important
F9	Alerter sur réservoir vide	Le niveau sous le seuil bloque tout arrosage et s'affiche sur le dashboard	Souhaitable

F5 porte la contrainte la plus forte du projet. L'arrêt de la pompe ne doit jamais dépendre de l'arrivée d'un second message : la durée est décidée au démarrage et appliquée par la carte embarquée, même si le réseau tombe entre-temps.

Architecture technique

L'architecture sépare trois couches, chacune sur le matériel qui lui convient. L'Arduino Yún gère le temps réel et l'analogique ; le Raspberry Pi 3 gère la persistance et la restitution.

```

flowchart LR
    DHT[DHT11<br/>temp + humidite] --> YUN
    LDR[Photoresistance<br/>luminosite] --> YUN
    EAU[Capteur<br/>niveau d eau] --> YUN
    YUN[Arduino Yun<br/>acquisition] -->|USB serie<br/>JSON| PI
    YUN --> RELAIS[Relais 5V]
    RELAIS --> POMPE[Pompe 12V]
    YUN --> LED[Eclairage<br/>rouge / bleu]
    PI[Raspberry Pi 3<br/>traitement] --> BDD[(SQLite)]
    BDD --> DASH[Dashboard web<br/>Flask]
    DASH -->|commandes| PI
    NAV[Navigateur<br/>sur le reseau local] --> DASH
  
```

Couche acquisition — Arduino Yún. Elle lit les trois capteurs, met en forme chaque mesure en JSON et commande les actionneurs. Elle porte les sécurités temps réel : durée maximale de pompage, refus de démarrer si le réservoir est vide. Ces règles sont dans le microcontrôleur et non dans le Pi, pour qu'elles s'appliquent même si la liaison tombe.

Couche traitement — Raspberry Pi 3. Un service Python reçoit les mesures, les insère dans la base SQLite et relaie les commandes d'arrosage et d'éclairage vers le Yún. Un tampon mémoire conserve les mesures si l'écriture en base échoue, et les réinsère ensuite.

Couche restitution — dashboard web. Une base SQLite stocke l'historique dans un fichier unique sur le Pi. Un dashboard servi par Flask le lit et l'affiche : c'est le centre de contrôle du système, accessible par un simple lien depuis un navigateur du réseau local.

La liaison Yún → Pi se fait en USB série. Le Yún dispose d'une interface WiFi et d'une API REST utilisable en repli, mais le câble supprime le WiFi de la liste des causes de panne — les deux cartes étant de toute façon côte à côte dans le même boîtier.

Interface de supervision

Le dashboard est le seul point de contact entre l'utilisateur et le système. Il est hébergé sur le Raspberry Pi et s'ouvre par un lien du type `http://raspberrypi.local:5000` depuis n'importe quel appareil du réseau local — aucune installation côté client.

Ce que la page affiche

- Les trois dernières mesures en gros caractères : température, humidité, luminosité
- Le niveau du réservoir, avec un état lisible d'un coup d'œil
- Les courbes des 24 dernières heures pour chaque grandeur
- L'état courant de la pompe et de l'éclairage
- La date de la dernière mesure reçue, pour repérer une chaîne d'acquisition muette

Ce que la page commande

- Un bouton d'arrosage, avec choix de la durée
- Un interrupteur d'éclairage
- Le journal des derniers arrosages, refus compris

Le dashboard ne décide de rien : il envoie des ordres et affiche des états. Les sécurités restent dans le microcontrôleur, de sorte qu'un navigateur fermé au mauvais moment ne puisse pas laisser la pompe en marche.

Technologies utilisées

Couche	Technologie	Rôle	Pourquoi ce choix
Embarqué	Arduino Yún (ATmega32u4 + AR9331)	Acquisition, commande des actionneurs	12 entrées analogiques, temps réel, et un côté Linux avec WiFi utilisable en repli
Embarqué	C++ / Arduino IDE	Programmation du sketch	Standard de la plateforme, bibliothèques capteurs disponibles
Embarqué	Bibliothèque DHT (Adafruit)	Lecture du protocole 1-fil du DHT11	Évite de réimplémenter un protocole à timings serrés
Embarqué	Bridge / YunServer	API REST exposée par le Yún	Permet de commander la pompe sans passer par le câble série
Serveur	Raspberry Pi 3 Model B / Raspberry Pi OS	Hébergement du service, de la base et du dashboard	Faible consommation, fonctionne en continu, largement documenté
Serveur	Python 3	Service de réception, de commande et serveur web	Écosystème riche, lisible, connu de l'équipe
Serveur	pyserial	Lecture du flux JSON venant du Yún	Bibliothèque de référence pour le port série
Serveur	requests	Appel de l'API REST du Yún	Commande de la pompe et de l'éclairage
Serveur	Flask	Serveur du dashboard et des routes de commande	Micro-framework : quelques dizaines de lignes suffisent, pas de configuration lourde sur un Pi 3
Serveur	systemd	Démarrage automatique et redémarrage des services	Le système doit repartir seul après une coupure de courant

Couche	Technologie	Rôle	Pourquoi ce choix
Client	HTML / CSS / JavaScript	Page du dashboard	Servie telle quelle, aucune installation côté utilisateur
Client	Chart.js	Courbes des 24 dernières heures	Bibliothèque légère, rendu côté navigateur, n'alourdit pas le Pi
Données	SQLite 3	Persistance des mesures et des arrosages	Base dans un fichier unique, sans serveur à administrer : adaptée à un seul écrivain et à un volume de quelques milliers de lignes par jour
Données	Module <code>sqlite3</code>	Accès à la base depuis Python	Inclus dans la bibliothèque standard, aucune dépendance à installer
Mécanique	SolidWorks	Modélisation du boîtier et des supports	Outil de CAO paramétrique utilisé en cours, export STL direct
Mécanique	Impression 3D FDM (PLA)	Fabrication des pièces	Coût faible, itération rapide entre deux versions

Le partage des rôles entre les deux cartes est le choix structurant. Le Yún seul ne sait pas stocker un historique ni servir une page web ; le Pi seul lit mal l'analogique et n'a pas de vrai temps réel. Les deux ensemble couvrent la chaîne, au prix d'une liaison à fiabiliser.

SQLite est retenue plutôt qu'un SGBD serveur parce que le système n'a qu'un seul processus écrivain et une charge de lecture minime. Le mode WAL doit être activé : il permet au dashboard de lire pendant que le service d'acquisition écrit, sans blocage.

Nomenclature matérielle et câblage

Élément	Référence	Qté	Caractéristiques utiles
Carte d'acquisition	Arduino Yún	1	5 V, 12 entrées analogiques, WiFi

Élément	Référence	Qté	Caractéristiques utiles
Carte serveur	Raspberry Pi 3 Model B	1	85,6 × 56,5 mm, alimentation 5 V microUSB
Température / humidité	DHT11 (4 broches)	1	0-50 °C, 20-90 % HR, résistance de tirage 10 kΩ
Luminosité	Photorésistance LDR	1	Diviseur de tension avec résistance 10 kΩ
Niveau d'eau	Module résistif à pistes	1	Sortie analogique, alimentation commutée
Pompe	Makeblock 370-04PM	1	12 V, < 320 mA en charge, 2 L/min
Commutation pompe	Module relais SRD-05VDC-SL-C	1	Bobine 5 V, contacts 10 A, optocoupleur intégré
Éclairage	LED de croissance rouge / bleu	1	Remplace la LED simple du prototype
Protection	Diode 1N4007	1	Roue libre aux bornes de la pompe
Prototypage	Breadboard demi-format	1	Interne au boîtier uniquement
Alimentations	Bloc 5 V (Pi) et bloc 12 V 1 A (pompe)	2	Deux circuits séparés

Affectation des broches de l'Arduino Yún

Broche	Usage	Sens
5 V	Alimentation partagée des trois capteurs	Sortie
GND	Masse commune des capteurs	—
D2	DATA du DHT11	Entrée numérique
A0	Signal de la photorésistance	Entrée analogique
A1	Signal du capteur de niveau	Entrée analogique
D8	Commande du relais de pompe	Sortie numérique
D9	Commande de l'éclairage	Sortie numérique

Les broches D0 et D1 restent libres : elles portent la liaison série interne avec le processeur Linux du Yún.

Deux points de câblage conditionnent la fiabilité. Le circuit 12 V de la pompe n'a **aucun point commun** avec le 5 V logique : les contacts du relais sont isolés, et c'est ce qui protège les cartes. Le capteur de niveau n'est **alimenté que pendant la mesure**, par une broche numérique, car un courant continu dans l'eau corrode ses pistes en quelques jours.

Conception mécanique

Le boîtier et les supports de capteurs sont modélisés sous SolidWorks, puis exportés en STL et imprimés en PLA.

Architecture du boîtier. Un socle de 260 × 180 × 24 mm divisé en trois baies : Raspberry Pi à gauche dans son boîtier ventilé, Arduino Yún et breadboard au centre, réservoir d'eau à droite. Une façade amovible ferme l'avant. Chaque sous-ensemble est vissable indépendamment, ce qui permet de démonter une carte sans tout ouvrir.

Contraintes d'impression prises en compte

- Parois de 2,4 mm minimum, jeu d'assemblage de 0,4 mm
- Fenêtres de connecteurs ouvertes vers le haut, pour imprimer sans support
- Avant-trous de 2,5 mm taraudés directement par des vis M3, sans insert
- Socle scindé en deux demi-socles assemblés par une cloison-bride et trois vis : 260 mm ne tiennent pas sur un plateau d'imprimante de 256 mm

Capteurs déportés. Le choix le plus structurant de la partie mécanique est de ne pas fixer les capteurs sur le boîtier. Un DHT11 monté sur le boîtier mesure l'air réchauffé par le Raspberry Pi, qui dissipe 2 à 4 W — soit un écart de 3 à 5 °C avec l'air réel de la plante, ce qui rend l'humidité relative inexploitable. Un capteur de lumière collé au boîtier mesure l'ombre du boîtier.

Les capteurs sont donc portés par un piquet de 195 mm planté dans le terreau : DHT11 en tête derrière une fente de ventilation, capteur de lumière à hauteur de feuillage, goulotte à câbles à l'arrière du fût. Une collerette fixe la profondeur d'enfoncement. La sonde de niveau est tenue par une pince réglable sur le bord du réservoir, avec seulement les pistes métalliques immergées.

Raccordement. Un bornier à vis dans la baie de droite reçoit la nappe venant du piquet. Les fils Dupont ne sortent jamais du boîtier : ce qui bouge et prend l'eau est vissé, ce qui est fragile reste à l'intérieur. La breadboard est conservée pour la phase de développement, et destinée à être remplacée par une plaque à pastilles soudée aux mêmes dimensions avant le rendu.

Modèle de données

La base est un fichier SQLite unique, `releves.db`, posé dans le répertoire du projet sur le Pi. Elle comporte deux tables. Le modèle est volontairement générique : une colonne `capteur` plutôt qu'une colonne par grandeur, pour qu'ajouter un capteur ne demande aucune migration.

Table `mesures`

Colonne	Type SQLite	Rôle
<code>id</code>	INTEGER PRIMARY KEY AUTOINCREMENT	Clé primaire
<code>capteur</code>	TEXT NOT NULL	Identifiant logique : <code>temperature</code> , <code>humidite</code> , <code>luminosite</code> , <code>niveau_cuve</code>
<code>valeur</code>	REAL NOT NULL	Valeur mesurée
<code>mesure_le</code>	TEXT NOT NULL	Instant de la mesure, au format ISO-8601
<code>recu_le</code>	TEXT NOT NULL DEFAULT (datetime('now'))	Instant d'insertion en base

Index sur (`capteur`, `mesure_le`) : le dashboard filtre toujours par capteur puis par période. Les dates sont stockées en texte ISO-8601, format que SQLite trie et compare correctement avec ses fonctions `date()` et `datetime()`.

La distinction entre `mesure_le` et `recu_le` n'est pas cosmétique. Quand le service réinsère des mesures après une coupure, l'écart entre les deux colonnes montre exactement combien de temps la liaison a été interrompue.

Table `arrosages`

Colonne	Type SQLite	Rôle
<code>id</code>	INTEGER PRIMARY KEY AUTOINCREMENT	Clé primaire
<code>demande_le</code>	TEXT NOT NULL	Instant de la commande, ISO-8601
<code>duree_s</code>	REAL NOT NULL	Durée demandée en secondes
<code>origine</code>	TEXT NOT NULL DEFAULT 'dashboard'	<code>dashboard</code> , <code>planifie</code> , <code>test</code>

Colonne	Type SQLite	Rôle
resultat	TEXT NOT NULL	Réponse de la carte : arrosage effectué, réservoir vide, repos non écoulé
niveau	INTEGER	Niveau d'eau relevé au moment de la commande

La colonne `resultat` journalise aussi les **refus**. Un arrosage bloqué parce que le réservoir était vide est une information à conserver : c'est elle qui explique, plus tard, pourquoi la plante n'a pas été arrosée.

Exigences non fonctionnelles et sécurité

Sécurité électrique. Le projet fait cohabiter de l'eau, un réservoir ouvert et deux circuits électriques. Trois règles s'appliquent sans exception : alimentation 12 V par un bloc secteur certifié et jamais par un montage maison ; aucune électronique en contact direct avec l'eau, seules les pistes métalliques de la sonde sont immergées ; isolation galvanique entre le 12 V de puissance et le 5 V logique, assurée par les contacts du relais.

Sécurité de l'arrosage. Trois garde-fous sont implémentés dans le microcontrôleur, pas dans le Pi.

1. Durée maximale de 20 s par cycle, appliquée même si aucun ordre d'arrêt n'arrive
2. Temps de repos minimal entre deux cycles, pour empêcher un enchaînement de commandes de noyer le pot
3. Refus de démarrer et coupure immédiate si le niveau passe sous le seuil, la pompe ne tournant jamais à sec

Le relais est câblé sur son contact normalement ouvert : carte non alimentée ou plantée, la pompe est à l'arrêt. L'état de repli du système est l'absence d'eau, jamais l'inverse.

Fiabilité des mesures. Chaque lecture analogique est une moyenne de huit échantillons, la première lecture après changement de canal étant écartée. Le capteur de niveau n'est alimenté que pendant la mesure pour limiter l'électrolyse. Une diode de roue libre aux bornes de la pompe évite que la surtension de coupure ne parasite les mesures voisines.

Disponibilité. Le service de réception et le serveur du dashboard sont lancés par systemd avec redémarrage automatique. La base SQLite est ouverte en mode WAL, pour que le dashboard puisse lire pendant que le service écrit. Les mesures sont tamponnées en mémoire si l'écriture échoue, puis réinsérées, et sauvegardées en CSV en dernier recours à l'arrêt. Objectif : aucune mesure perdue sur une coupure de 5 minutes.

Sécurité d'accès. SQLite n'a pas de comptes utilisateurs : la protection repose sur les droits du fichier `reLeves.db`, lisible et modifiable par le seul compte qui fait tourner les services. Le dashboard n'écoute que sur le réseau local et n'est pas exposé sur Internet. Ses routes de commande n'acceptent que des requêtes POST, pour qu'un lien visité par erreur ne puisse pas déclencher un arrosage. Une sauvegarde périodique du fichier de base est à prévoir avant le rendu — un fichier unique se perd aussi facilement qu'il se copie.

Risques identifiés et points à valider

Risque	Impact	Parade prévue
Effet siphon : l'eau continue de couler après l'arrêt de la pompe	Élevé — le réservoir se vide dans le pot	Point haut du tuyau au-dessus du niveau d'eau, ou trou d'air de 1 mm au sommet de la boucle
Précision du DHT11 ($\pm 2^\circ\text{C}$, $\pm 5\%$ HR)	Moyen — mesures peu fines	Assumé pour ce rendu ; DHT22 en remplaçant direct si le coach le demande
Corrosion de la sonde de niveau	Moyen — dérive puis panne	Alimentation commutée, recalibrage prévu avant le rendu
Fils Dupont qui se débranchent	Élevé — valeurs fantaisistes sans erreur visible	Bornier à vis pour tout ce qui sort du boîtier, plaque à pastilles avant le rendu
Impression 3D ratée ou hors dimensions	Moyen — retard de quelques heures	Pièce test imprimée en premier, cotes de la breadboard et du réservoir à remesurer
Consommation de l'éclairage sous-évaluée	Moyen — alimentation insuffisante	Puissance des LED à mesurer avant de figer le bloc d'alimentation
Liaison série interrompue	Faible — mesures tamponnées	Reconnexion automatique, tampon mémoire, redémarrage par <code>systemd</code>

Points à trancher avec le coach

- **Éclairage.** La LED simple du prototype doit être remplacée par des LED de croissance rouge / bleu. Leur puissance, leur tension d'alimentation et leur mode de commande (tout ou rien, ou variation par PWM) ne sont pas arrêtés. Au-delà de quelques centaines de milliampères, elles ne se pilotent plus depuis une broche et demandent un transistor ou un second relais.
- **Humidité du substrat.** Elle est actuellement hors périmètre, alors que c'est la grandeur la plus pertinente pour décider d'un arrosage. Un capteur capacitif coûte peu et se branche sur A2. À arbitrer selon le temps disponible.
- **Profondeur du dashboard.** L'affichage des mesures et les deux boutons de commande sont acquis. Reste à trancher ce qui s'y ajoute : rafraîchissement automatique, plage d'historique réglable, export CSV. Chaque option est un lot de travail à part.
- **Autonomie de démonstration.** Faire tourner le système plusieurs jours avant la soutenance change la nature de la démonstration : le dashboard montre alors un historique réel plutôt qu'une poignée de points.

Planning et jalons

Le cahier des charges reste ouvert pendant les deux premiers jours. Les retours du coach et les contraintes de faisabilité rencontrées au montage peuvent en déplacer le périmètre — la section Risques liste les arbitrages en attente.

Jalon	Contenu	État
J1-J2	Cadrage : cahier des charges, validation du périmètre avec le coach, arbitrage éclairage et humidité du substrat	En cours
J3	Acquisition : câblage des trois capteurs sur breadboard, sketch de lecture, valeurs cohérentes au moniteur série	À faire
J4	Persistance : base SQLite créée, service de réception sur le Pi, premières lignes insérées	À faire
J5	Dashboard : serveur Flask, affichage des dernières mesures puis des courbes	À faire
J6	Actionneurs : relais, pompe, circuit hydraulique, sécurités logicielles, éclairage, boutons du dashboard	À faire
J7	Mécanique : impression des pièces, montage, passage au bornier à vis	À faire

Jalon	Contenu	État
J8	Intégration : mise en service continue, calibrage des seuils, constitution d'un historique	À faire
J9	Rendu : dossier technique finalisé, démonstration répétée	À faire

L'ordre n'est pas négociable sur un point : la chaîne capteur → base doit fonctionner avant qu'on touche à la pompe. Un système qui arrose sans savoir mesurer n'a rien à démontrer.

Critères de validation et évolutions

Ce qui sera démontré au rendu

- ☐ Les trois capteurs remontent des valeurs cohérentes, vérifiables en direct (main sur le capteur de lumière, souffle sur le DHT11, sonde sortie de l'eau)
- ☐ Le dashboard s'ouvre depuis un téléphone ou un portable du réseau, et affiche les mesures à jour
- ☐ Les courbes des 24 dernières heures s'affichent à partir de données réelles
- ☐ Un bouton du dashboard déclenche l'arrosage, qui s'arrête seul à la durée demandée
- ☐ Le réservoir vidé bloque l'arrosage, et le refus apparaît dans le journal
- ☐ L'éclairage s'allume et s'éteint depuis le dashboard
- ☐ Le système redémarre seul après coupure de l'alimentation du Pi, dashboard compris
- ☐ Le boîtier imprimé est monté, câblé et transportable

Le dernier point mérite d'être testé pour lui-même. Un montage qui fonctionne sur la table mais tombe en panne quand on le déplace ne passe pas une soutenance : le transport jusqu'à la salle fait partie de la démonstration.

Évolutions envisagées

- Capteur d'humidité du substrat, pour déclencher l'arrosage sur la grandeur réellement pertinente
- Arrosage planifié depuis le dashboard, avec fenêtres horaires et débit journalier maximal
- Photopériode automatique sur l'éclairage, avec cycle jour / nuit paramétrable
- Notification en cas de réservoir vide ou de capteur muet depuis plus d'une heure
- Export CSV de l'historique depuis le dashboard
- Migration vers un SGBD serveur si le volume ou le nombre de lecteurs dépasse ce que SQLite couvre confortablement
- Remplacement du DHT11 par un DHT22 ou un BME280 si la précision devient un enjeu

Bilan de réalisation

Supervision Grafana et alertes e-mail

SPACEFARM 2080 | Pilier 2 : FoodTech & AgriTech | 24 septembre 2026

Travail réalisé. J’ai mis en place Grafana sur le Raspberry Pi 3 pour superviser les mesures de SpaceFarm et un système d’alertes automatiques par e-mail. L’objectif est d’être averti d’un problème sans devoir garder le dashboard ouvert.

1. Supervision ajoutée

Grafana lit l’API SpaceFarm existante : température, humidité, luminosité relative et niveau d’eau estimé. Il affiche leurs courbes sur les six dernières heures, l’état de la LED, le mode de commande et l’état du Pi. Le rafraîchissement est réglé à 30 secondes. La luminosité et le réservoir sont exprimés en pourcentage.

Le centre de contrôle existant reste utilisé pour commander la LED. Grafana sert à la consultation et à la surveillance ; il ne commande pas la pompe.

2. Alertes e-mail configurées

Condition surveillée	Délai de confirmation
Niveau d’eau estimé inférieur à 20 %	5 minutes
Température inférieure à 18 °C ou supérieure à 30 °C	5 minutes
Au moins une mesure invalide ou périmée	2 minutes

Les règles sont évaluées chaque minute. L’API considère une mesure périmée après 120 secondes sans actualisation. Après déclenchement, un délai de regroupement de 30 secondes s’applique. Un rappel est prévu toutes les 4 heures si le problème persiste, puis un message signale le retour à la normale.

3. Mise en œuvre et vérification

Chaîne : capteurs → Arduino Yún → API du Raspberry Pi → Grafana → serveur SMTP Gmail → boîte e-mail.

L’envoi utilise une connexion chiffrée STARTTLS avec vérification du certificat et un mot de passe d’application conservé hors du code. **Test effectué le 24/09/2026 : Grafana a confirmé le succès de l’envoi d’un e-mail de test.** La réception dans la boîte du destinataire reste à confirmer par son utilisateur.

4. Limites du prototype

Le Pi doit rester allumé et connecté à Internet pour expédier les messages. L’alerte de niveau d’eau dépend de la calibration de la sonde. Les seuils choisis servent à la démonstration et pourront être adaptés à la culture.

Mise à jour du cahier des charges : la notification annoncée parmi les évolutions en page 13 est désormais réalisée avec les conditions ci-dessus. Cette annexe décrit cet ajout ; elle ne valide pas les autres objectifs du document initial.